

Informe de la pràctica 3 Wifi i Bluetooth

Introducció

Aquesta tercera pràctica de l'assignatura fa un pas endavant en la complexitat i les capacitats del microcontrolador ESP32-S3, abandonant temporalment el control físic de components electrònics bàsics (com LEDs o botons) per endinsar-se en l'ús de perifèrics especialitzats de comunicació sense fils. L'exercici se centra a dotar la placa de connectivitat cap a l'exterior mitjançant l'ús de les tecnologies Wi-Fi i Bluetooth.

1.1. Objectius

L'objectiu principal d'aquesta pràctica és comprendre el funcionament i la implementació de protocols de xarxa Wi-Fi i Bluetooth en sistemes encastats. Per assolir-ho de manera pràctica i esglaonada, s'han definit els següents objectius específics:

- **Generació de servidors web (Wi-Fi STA):** Configurar l'ESP32 perquè es connecti a una xarxa Wi-Fi existent com a "Estació" i aixequi un servidor HTTP capaç d'allotjar i enviar codi HTML a qualsevol navegador web que ho sol·liciti.
- **Comunicació Bluetooth bidireccional:** Establir un pont de comunicació sèrie inalàmbic entre el microcontrolador i una aplicació d'un telèfon mòbil intel·ligent utilitzant Bluetooth Clàssic.
- **Actualització de firmware sense fils (OTA):** Implementar la tecnologia *Over-The-Air* per poder enviar nous programes a la placa a través de la xarxa local Wi-Fi, eliminant la dependència del cable USB. (Pràctica Complementària 1).
- **Gestió avançada de xarxes i fitxers:** Integrar un servidor web asíncron vinculat al sistema de fitxers intern de la memòria flaix (SPIFFS) per crear un gestor d'arxius remot robust i eficient. (Pràctica Complementària 2).

1.2. Material utilitzat

Per dur a terme aquesta pràctica i poder verificar la connectivitat, el muntatge no requereix components electrònics externs al xip, però sí que es fa ús de maquinari i programari de xarxa:

Maquinari (Hardware):

- **Microcontrolador:** Placa de desenvolupament basada en el xip ESP32-S3, el qual integra antenes i mòduls físics de Wi-Fi i Bluetooth de sèrie.
- **Infraestructura de xarxa:** Un router domèstic per proporcionar la xarxa Wi-Fi local ("Nautilus").
- **Dispositiu Client:** Un telèfon mòbil intel·ligent (*smartphone*) o un ordinador per connectar-se als serveis generats per la placa.

Programari (Software):

- **Entorn de treball:** Visual Studio Code amb l'extensió PlatformIO (Framework Arduino en C++).
- **Aplicació Mòbil:** L'aplicació "Serial Bluetooth Terminal" (desenvolupada per Kai Morich), utilitzada com a eina de monitoratge i enviament de dades via Bluetooth des del telèfon.
- **Llibreries de tercers:** Llibreries especialitzades descarregades des de GitHub ([AsyncTCP](#) i [ESPAsyncWebServer](#)) per a la creació de servidors no bloquejants.

1.3. Conceptes Claus: Connectivitat i Protocols

Tal com s'indica a la introducció teòrica de la pràctica, l'ús de perifèrics Wi-Fi i Bluetooth es basa en una sèrie de protocols i estàndards de xarxes que operen "per sota" del nostre codi:

- **Pila TCP/IP i HTTP:** Perquè la placa pugui mostrar una pàgina web, no només s'ha de connectar a la freqüència de ràdio del router (Wi-Fi), sinó que ha de rebre una adreça IP vàlida. A sobre d'aquesta IP, el protocol HTTP actua com un missatger estructurat que transporta el text HTML des del servidor (l'ESP32) fins al client (el navegador) de forma comprensible.
- **Bluetooth SPP (Serial Port Profile):** En la Pràctica B, el Bluetooth no s'utilitza per transmetre àudio ni fitxers complexos, sinó que es configura com un perfil de port sèrie (SPP). Això significa que el sistema emula de forma sense fils la mateixa consola o terminal que normalment tindriem connectada a través del cable USB, facilitant enormement l'intercanvi de missatges de text i ordres directes amb el dispositiu.

Desenvolupament i Implementació (Pràctica A)

Aquesta tercera pràctica introdueix l'ús de perifèrics de comunicació sense fils. A la primera part (Pràctica A), l'objectiu és configurar l'ESP32 perquè es connecti a una xarxa Wi-Fi existent (Mode *Station* o STA) i actuï com a servidor web, capaç d'enviar una pàgina HTML a qualsevol dispositiu (com un mòbil o un ordinador) que s'hi connecti mitjançant un navegador.

1. Configuració de l'Entorn ([platformio.ini](#))

El fitxer de configuració manté la definició de la placa i l'entorn de treball ([espressif32](#), [esp32-s3-devkitm-1](#), [arduino](#)).

- **[monitor_speed = 115200](#):** En aquesta pràctica, configurar correctament la velocitat del terminal sèrie a 115200 bauds és absolutament vital. Com veurem a continuació, el router assignarà una adreça IP dinàmica a la nostra placa, i l'única manera de saber quina és per poder-nos-hi connectar des del navegador serà llegint-la a través d'aquest terminal.

2. Codi Principal: Pràctica A - Generació d'una pàgina web (**main.cpp**)

Aquest codi s'encarrega d'establir la connexió Wi-Fi i de posar el microcontrolador a l'escolta de peticions HTTP externes.

Llibreries i Configuració de Xarxa:

- **#include <WiFi.h>** i **#include <WebServer.h>**: Importen les llibreries específiques per gestionar el maquinari Wi-Fi de l'ESP32 i per crear l'estructura del servidor web.
- **const char* ssid = "Nautilus";** i **const char* password = "2... ";** Es defineixen les credencials (nom de la xarxa i contrasenya) del router al qual ens volem connectar.
- **WebServer server(80);**: Es crea l'objecte del servidor assignant-li el port **80**. Aquest és el port estàndard per defecte per a les comunicacions HTTP de les pàgines web.

Estructura de la Pàgina Web (HTML):

- **String HTML = R"====(...)====";**: Aquesta és una forma molt elegant de definir el contingut de la web. L'ús de literals de cadena sense processar (**R"====(...)===="**) permet escriure codi HTML complet en múltiples línies (amb les seves etiquetes **<!DOCTYPE html>**, **<html>**, **<body>**, **<h1>**, etc.) sense haver de posar cometes complexes a cada línia o concatenar strings contínuament. Aquest apartat dóna compliment a la petició de la pràctica de modificar i generar un fitxer HTML.

Funció d'Enrutament:

- **void handle_root() { server.send(200, "text/html", HTML); }**: Aquesta funció s'executarà quan un client demani l'arrel de la web (/). La instrucció **server.send** retorna un codi **200** (que en el protocol HTTP significa "OK / Èxit") i envia al navegador el text HTML que hem dissenyat anteriorment.

Funció **setup()** (Connexió):

- **WiFi.begin(ssid, password);**: Ordena al microcontrolador que iniciï la negociació amb el router per connectar-se a la xarxa especificada.
- **while (WiFi.status() != WL_CONNECTED) { delay(1000); Serial.print("."); }**: Aquest és un bucle d'espera. Com que la connexió Wi-Fi pot trigar uns segons en establir-se i rebre una IP, el codi es queda aquí aturat ("bloquejat" temporalment de forma intencionada) imprimint un punt cada segon fins que l'estat canvia a connectat.
- **WiFi.localIP()**: Obté l'adreça IP local que el router ha atorgat a l'ESP32.
- **server.on("/", handle_root);** i **server.begin();**: Associa la ruta principal ("/") amb la funció **handle_root** que hem vist abans i posa el servidor a funcionar.

Funció `loop()`:

- `server.handleClient()`;: El bucle principal es manté lliure de condicions complexes. Aquesta única instrucció s'encarrega d'escoltar de forma contínua si hi ha algun client extern (un telèfon, un PC) que està intentant accedir a la IP del servidor.

Sortida pel Terminal de Control

Per verificar el correcte funcionament de la Pràctica A, la sortida del monitor sèrie juga un paper decisiu. En arrancar l'ESP32, es visualitza la següent seqüència:

1. **Intentando conectar a Nautilus**
2. A continuació, imprimeix punts (. . .) durant uns segons mentre el xip negocia amb el router.
3. Un cop superat el procés, s'imprimeix l'èxit de l'operació i, **el més important**, l'adreça IP on s'allotja la pàgina: **WiFi conectado exitosamente Got IP: 192.168.1.XX** (Aquesta dada depèn del router)
4. Finalment, avisa que està a l'escolta amb: **Servidor HTTP iniciado.**

Si agafem la IP proporcionada al terminal (per exemple, **192.168.1.XX**) i la introduïm a la barra de direccions de qualsevol navegador web connectat a la mateixa xarxa Wi-Fi, apareixerà per pantalla el text modificat en HTML amb el títol "*Mi Primera Pagina con ESP32 Station Mode*".

Anàlisi del Temps Lliure del Processador (Servidor Web)

L'anàlisi de l'ocupació de la CPU en aquesta pràctica ens revela un comportament híbrid que depèn de la fase en què es trobi el programa: la fase d'inicialització o la fase de servei.

Fase d'Inicialització (Setup): Durant els primers segons d'execució, el processador dedica gairebé el 100% del seu temps a intentar establir la connexió amb el router. Utilitza un bucle `while` amb una instrucció `delay(1000)`. Com ja sabem, això és altament bloquejant. No obstant això, en el context d'arrencada d'un dispositiu de xarxa, aquest bloqueig és acceptable (i sovint necessari), ja que el sistema no pot oferir cap servei web fins que no tingui una adreça IP vàlida assignada.

Fase de Servei (Loop) i el mètode de Polling: Un cop connectat, el programa entra en el bucle principal, on executa exclusivament la instrucció `server.handleClient()`; . A diferència de la Pràctica 2 (on va utilitzar interrupcions per no haver de preguntar si el botó s'havia premut), la llibreria estàndard `WebServer` funciona basant-se en el mètode de **consulta contínua o polling**.

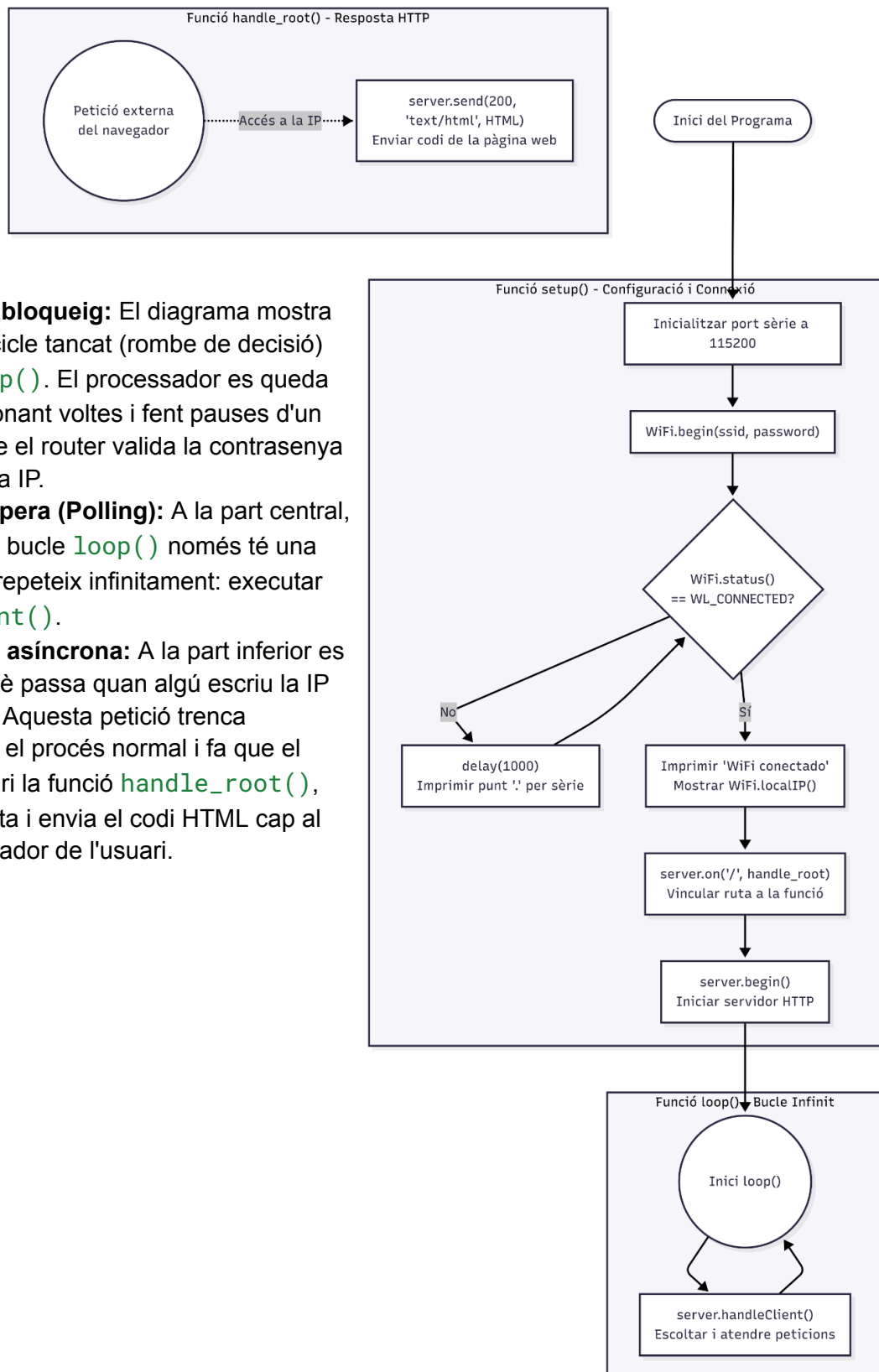
Això significa que el microcontrolador està executant aquest bucle milers de vegades per segon, preguntant contínuament al xip Wi-Fi: *"Hi ha algun client nou intentant connectar-se?"*.

- **Temps de resposta a la petició:** Quan finalment un usuari introdueix la IP al navegador, la funció `handle_root()` entra en acció. Atès que només ha d'enviar una cadena de text (l'HTML) de pocs bytes, aquesta operació triga només uns pocs mil·lisegons abans de tornar al bucle d'espera.
- **Ocupació de la CPU:** Tot i que no hi ha cap `delay()` bloquejant al `loop()`, el processador **no està adormit ni té temps lliure real**, ja que està constantment treballant en el procés de *polling* de clients de xarxa.

Conclusió: Des de la perspectiva de l'usuari i de l'aplicació, el servidor passa el 99,9% del temps "lliure" esperant que algú sol·liciti la pàgina web. Tanmateix, des de la perspectiva de l'arquitectura del microcontrolador, està treballant al 100% executant el bucle de consulta `server.handleClient()`. En aplicacions d'Internet de les Coses (IoT) molt avançades on l'estalvi de bateria és crític, se solen utilitzar llibreries de servidors web *asíncrons* (basats en interrupcions) per poder posar la CPU a dormir durant tot aquest temps d'espera.

Diagrames de flux i temps

Diagrama de flux

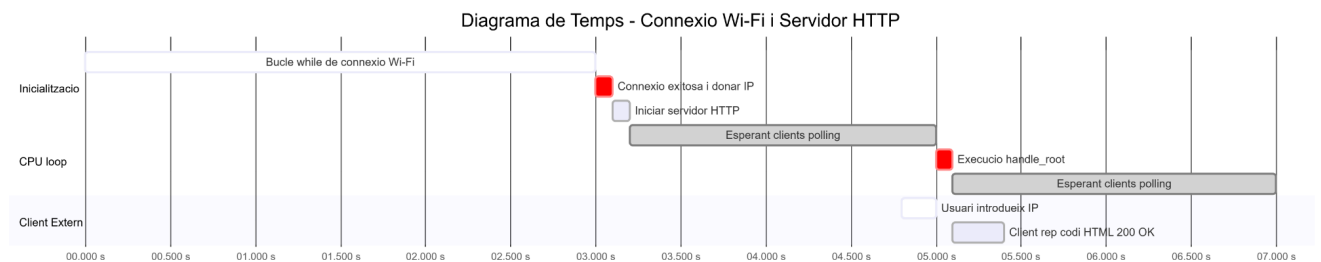


El procés de bloqueig: El diagrama mostra clarament el cicle tancat (rombe de decisió) dins del `setup()`. El processador es queda atrapat allà donant voltes i fent pauses d'un segon fins que el router valida la contrasenya i li assigna una IP.

El mode d'espera (Polling): A la part central, es veu com el bucle `loop()` només té una tasca que es repeteix infinitament: executar `handleClient()`.

La interacció asíncrona: A la part inferior es representa què passa quan algú escriu la IP al navegador. Aquesta petició trenca temporalment el procés normal i fa que el servidor dispari la funció `handle_root()`, que empaqueta i envia el codi HTML cap al mòbil o l'ordinador de l'usuari.

Diagrama de temps



Fase de Bloqueig Inicial (0 a 3 segons): La primera franja mostra l'execució del `setup()`. A diferència d'altres pràctiques, aquí el processador passa la major part del temps aturat dins del bucle `while`, esperant que el maquinari Wi-Fi acabi de negociar amb el router de casa i li assigni una adreça IP.

Fase de Servei i Polling (a partir del segon 3,2): Un cop el servidor està iniciat, la CPU entra al `loop()` (franja central). El processador executa contínuament la instrucció `server.handleClient()`. Està "esperant" clients (representat en gris), però treballant a ple rendiment fent comprovacions de xarxa.

Esdeveniment Extern i Resposta: Al segon 5, un usuari extern (franja inferior) demana accedir a la IP des del seu navegador mòbil o PC. Immediatament, l'ESP32 detecta la petició al seu bucle de *polling*, atura temporalment l'escolta per executar la funció `handle_root()` (marcat en vermell), i envia tot el codi HTML al client gairebé a l'instant. Acte seguit, el bucle torna al seu estat d'espera.

Desenvolupament i Implementació: Pràctica Complementària 1 (Actualització OTA)

Aquest exercici ha consistit a configurar el microcontrolador perquè sigui capaç de rebre i instal·lar nous programes sense fils a través de la xarxa Wi-Fi local. Aquest procés requereix una configuració especial en dues etapes: una primera càrrega per cable per preparar el xip i una segona càrrega sense fils.

1. Configuració de l'Entorn (`platformio.ini`)

Per a la primera fase (càrrega per cable), el fitxer manté la configuració estàndard amb el `monitor_speed = 115200`. No obstant això, per dur a terme la segona fase (l'enviament des del portàtil via Wi-Fi), s'ha hagut de modificar el fitxer `.ini` afegint dues línies fonamentals:

- **`upload_protocol = espota`:** Aquesta instrucció canvia el comportament per defecte de l'entorn de programació. Indica a PlatformIO que no ha de buscar un port sèrie/USB per carregar el codi, sinó que ha d'utilitzar el protocol de xarxa *Espressif Over-The-Air*.

- **upload_port = 192.168.50.102**: Un cop estem en mode xarxa, aquesta línia indica la direcció IP exacta cap a on s'ha d'enviar el codi compilat. Aquesta IP és la que l'ESP32 ha obtingut del router en la primera fase.

2. Codi Principal: Fases d'Execució OTA (**main.cpp**)

L'arxiu principal està clarament dividit en dues parts que il·lustren el flux de treball d'aquesta tecnologia.

Fase 1: Preparació del xip (Primer codi) Aquest codi s'ha de pujar inicialment per cable per dotar la placa de connectivitat.

- **ArduinoOTA.begin();**: Després de connectar-se a la Wi-Fi "Nautilus", aquesta línia encén el servei OTA oficial a l'interior del xip i obre un port de xarxa intern perquè estigui a punt per rebre arxius.
- **ArduinoOTA.handle();**: Situada dins del bucle principal (**loop**), aquesta funció s'executa contínuament. La seva feina és escoltar de forma ininterrompuda la xarxa, esperant que algun ordinador amb la IP de l'ESP32 iniciï una transferència de codi.

Fase 2: El programa enviat via Wi-Fi Un cop configurat el **.ini** amb la IP, s'ha enviat el segon bloc de codi sense fils.

- **Gestió del temps no bloquejant (millis())**: Per tal de demostrar que el nou codi s'ha executat, s'ha programat l'enviament d'un missatge per port sèrie. La part clau de la implementació és que s'ha utilitzat la tècnica **if (millis() - tiempoAnterior > 2000)** en comptes d'un **delay(2000)**.
- L'ús de la funció **delay()** està terminantment prohibit en el bucle principal d'un codi OTA, ja que paralitzaria l'execució de **ArduinoOTA.handle()** durant l'espera. Si la placa no escolta el servei d'actualització de forma contínua i lliure, el xip perdria la capacitat de rebre futures actualitzacions sense fils.

Sortida pel Terminal de Control

En aquest exercici, la sortida pel terminal mostra l'evolució de les dues fases de la pràctica:

1. En la preparació inicial (per cable), el terminal mostra la connexió a la xarxa i subministra la IP necessària per configurar el PlatformIO: **WiFi connected IP address: 192.168.50.102**
2. Un cop feta la càrrega sense fils ("*upload*") des del portàtil, el microcontrolador es reinicia automàticament i passa a executar el segon codi. A partir d'aquest moment, el terminal imprimeix de forma periòdica i exacta cada 2 segons: **¡Actualizacion por Wi-Fi completada con exito! Version 2.0**

Anàlisi del Temps Lliure del Processador (Actualització OTA)

L'anàlisi de l'ocupació de la CPU en aquest programa demostra que disposar de "temps lliure" no és només una qüestió de rendiment, sinó un **requisit tècnic obligatori** perquè el servei OTA funcioni correctament.

L'arrencada i el bloqueig justificat: Igual que a la Pràctica A, durant la fase d'arrencada (`setup()`), el processador es troba totalment ocupat i bloquejat per la instrucció `delay(500)` dins del bucle de connexió Wi-Fi. Això és necessari, ja que no té sentit iniciar el servei OTA abans de tenir una adreça IP vàlida a la xarxa.

El Bucle Principal (Loop) i l'exigència de temps real: Un cop dins del bucle `loop()`, el comportament ha de ser extremadament fluid.

- **La instrucció `ArduinoOTA.handle()`:** Aquesta funció s'ha d'executar el màxim nombre de vegades possible per segon. La seva tasca és comprovar ràpidament si hi ha algun paquet de xarxa entrant que sol·liciti una actualització. Si no n'hi ha, la funció acaba gairebé a l'instant.
- **L'ús de `millis()` vs `delay()`:** Per demostrar l'execució del programa, calia enviar un missatge cada 2 segons. Si s'hagués utilitzat `delay(2000)`, el processador s'hauria aturat completament, deixant d'executar `ArduinoOTA.handle()` durant aquells 2 segons. Això hauria provocat que qualsevol intent de carregar un codi des del portàtil fos ignorat o donés un error de temps d'espera (*timeout*). L'ús de `if (millis() - tiempoAnterior > 2000)` soluciona aquest problema.

Distribució del temps i ocupació:

- **Avaluació de condicions (< 1%):** El processador només gasta una fracció de mil·lisegon en avaluar si ja han passat 2 segons i en revisar la xarxa a través del `handle()`.
- **Temps lliure d'espera activa (> 99%):** Atès que no hi ha cap `delay()`, el bucle gira a màxima velocitat. Tot aquest "temps lliure" permet que l'escolta del servei d'actualitzacions sigui contínua i sense talls. Només un cop cada 2 segons perd un instant per enviar la frase per comunicació sèrie.

Conclusió: En sistemes connectats (*IoT*) amb capacitat d'actualització remota, la gestió del temps passa a ser crítica. Aquest programa demostra que una arquitectura no bloquejant (utilitzant `millis()` per planificar tasques) allibera pràcticament el 100% de la CPU. Aquesta disponibilitat constant és precisament el que manté viu l'enllaç de xarxa en segon pla, permetent que l'ESP32 estigui sempre preparat per rebre un nou firmware de manera inalàmbrica.

Desenvolupament i Implementació: Pràctica Complementària 2 (Servidor Asíncron i Gestor d'Arxius)

Aquest exercici ha elevat la complexitat de la comunicació web implementada a la Pràctica A. S'ha substituït el servidor web estàndard per un de tipus **asíncron** i s'ha habilitat l'ús del sistema de fitxers SPIFFS (*Serial Peripheral Interface Flash File System*). Això converteix l'ESP32 en una petita memòria d'emmagatzematge en xarxa (NAS) accessible des de qualsevol dispositiu connectat a la Wi-Fi.

1. Configuració de l'Entorn (**platformio.ini**)

Per poder fer servir l'arquitectura asíncrona i gestionar les peticions d'una manera no bloquejant, s'ha hagut de modificar significativament l'arxiu de configuració instal·lant llibreries externes de tercers:

- **lib_deps = ...**: Aquest bloc és la gran novetat. Li diu a PlatformIO que, abans de compilar el codi, ha de descarregar i incloure automàticament dues llibreries des de GitHub:
 - <https://github.com/me-no-dev/AsyncTCP.git>: És el motor que gestiona les connexions de xarxa en segon pla (basades en interrupcions) en lloc d'utilitzar el mètode de *polling* tradicional.
 - <https://github.com/me-no-dev/ESPAsyncWebServer.git>: És la llibreria que construeix el servidor web asíncron aprofitant el motor TCP esmentat.

2. Codi Principal: El Servidor SPIFFS (**main.cpp**)

Aquest programa es divideix en la construcció de la interfície d'usuari i l'enrutament de les funcions del servidor.

Interfície d'Usuari i Javascript:

- **const char index_html[] PROGMEM = R"rawliteral(...)rawliteral";**: La pàgina web dissenyada per a aquest exercici és molt més complexa que la de la Pràctica A. Inclou codi JavaScript avançat que s'encarrega d'executar peticions internes per demanar la llista d'arxius al servidor (`fetch('/list')`) i de gestionar l'enviament de fitxers (`fetch('/upload')`) sense necessitat de recarregar la pàgina contínuament.

Inicialització i Sistema de Fitxers (**setup()**):

- **if(!SPIFFS.begin(true)) { ... }**: Aquesta línia és fonamental. Inicialitza el sistema de fitxers intern a la memòria Flash. El paràmetre `true` assegura que, si és la primera vegada que s'utilitza i no té el format correcte, el microcontrolador

formatejarà la partició automàticament perquè puguem començar a guardar-hi coses.

Rutes de l'API (Endpoints): El servidor respon de manera diferent depenent de la petició (ruta) que faci el JavaScript del navegador:

- **`server.on("/list", HTTP_GET, ...)`:** Quan el navegador demana l'arrel `/list`, el codi obre l'arrel de la memòria Flash (`SPIFFS.open("/")`), llegeix el nom de tots els arxius que hi ha guardats i construeix manualment una cadena de text en format JSON per enviar-la de tornada al navegador.
- **`server.on("/upload", HTTP_POST, ...)`:** Aquesta funció s'activa quan l'usuari clica al botó de "Subir Archivo". El servidor rep les dades de l'arxiu en petits fragments. Va ajuntant aquests trossos (`request->_tempFile.write(data, len);`) i els guarda directament a la memòria SPIFFS del xip fins a rebre la confirmació final.
- **`server.on("/download", HTTP_GET, ...)`:** Aquesta ruta rep un paràmetre (el nom de l'arxiu). Si l'arxiu existeix, n'ordena la descàrrega cap al mòbil o PC de l'usuari declarant-lo com un tipus d'arxiu genèric (`"application/octet-stream"`) per forçar que el navegador el descarregui en lloc d'intentar obrir-lo.

La gran diferència: La Funció `loop()`:

- A l'última línia del programa observem que la funció principal `loop()` **està completament buida**. Això demostra l'èxit de l'arquitectura asíncrona: el servidor web ja no necessita la instrucció `server.handleClient()` per comprovar si hi ha peticions (com feia a la Pràctica A). Tot el trànsit web es gestiona "per darrere" mitjançant interrupcions associades directament al maquinari de xarxa.

Sortida pel Terminal de Control

Durant l'execució d'aquesta aplicació, el terminal es fa servir principalment per fer el seguiment de les accions de l'usuari des del navegador:

1. En arrancar, es mostra el procés d'inicialització i l'adreça d'accés (per exemple: **IP del Gestor de Archivos: 192.168.1.XX**).
2. Quan l'usuari puja un arxiu (per exemple, un document anomenat `practica.txt`), el terminal mostra el progrés: **Iniciando subida: practica.txt Subida finalizada: practica.txt (1452 bytes)**
3. Si l'usuari clica a la llista per descarregar un arxiu, el servidor ho notifica: **Solicitando: /practica.txt**

Anàlisi del Temps Lliure del Processador (Servidor Asíncron)

El salt evolutiu d'aquesta pràctica respecte al servidor web estàndard (Pràctica A) és absolut. L'anàlisi de l'ocupació de la CPU ens mostra el màxim nivell d'eficiència possible.

L'eliminació del mètode de Polling: Com vam veure a la Pràctica A, la CPU estava obligada a executar contínuament la instrucció `server.handleClient()` per preguntar si algun usuari volia connectar-se. En aquest nou codi, gràcies a l'ús de la llibreria `ESPAsyncWebServer` recolzada per `AsyncTCP`, aquesta necessitat de consulta contínua desapareix per complet.

El Loop Buit (Temps Lliure Total): La prova irrefutable d'aquesta eficiència estructural es troba al final del programa: la funció `loop()` ha quedat completament buida. Això significa que el fil d'execució principal del processador gaudeix, a la pràctica, d'un **100% de temps lliure** mentre espera que els usuaris interactuïn amb el Gestor d'Arxius.

Gestió "a l'ombra" (Interrupcions i RTOS): Si el bucle principal està buit, com és que el servidor pot llistar, pujar i descarregar fitxers? L'arquitectura asíncrona delega tota la gestió de la xarxa Wi-Fi a les interrupcions de maquinari i a les tasques en segon pla del sistema operatiu intern de l'ESP32 (FreeRTOS).

- Quan l'usuari clica al botó de pujar un arxiu, l'antena Wi-Fi rep el senyal i genera una interrupció.
- La CPU processa aquesta petició en segon pla cridant directament a la funció associada a la ruta `/upload` i escrivint les dades a la memòria SPIFFS.
- Un cop acabada la transferència, la CPU torna al seu estat de disponibilitat total gairebé a l'instant.

Conclusió: Aquest model de servidor web asíncron representa l'estàndard professional per a dispositius de l'Internet de les Coses (IoT). L'alliberament total de la funció `loop()` permetria a aquest mateix ESP32 executar tasques industrials crítiques (com llegir sensors d'alta velocitat, controlar motors pas a pas o gestionar pantalles) sense que una tasca lenta com la descàrrega d'un arxiu per Wi-Fi bloquegi el microcontrolador o provoqui retards en el control físic del maquinari.

Conclusions

Aquesta tercera pràctica ha representat un punt d'inflexió en l'aprenentatge de l'arquitectura de microcontroladors, endinsant-nos de ple en el món de les comunicacions sense fils i l'Internet de les Coses (IoT). S'ha demostrat amb èxit la gran versatilitat del xip ESP32-S3 com a dispositiu connectat. De les tasques realitzades, se n'extreuen les següents conclusions fonamentals:

- **Implementació dual Wi-Fi i Bluetooth:** S'ha comprovat la facilitat amb què el microcontrolador pot establir connexions de xarxa tradicionals (Wi-Fi STA) per servir pàgines web, al mateix temps que pot emular perfils de port sèrie tradicionals a través de Bluetooth Clàssic (SPP) per comunicar-se de manera directa amb aplicacions mòbils.
- **L'evolució dels Servidors Web (Polling vs. Asíncron):** La pràctica ha permès viure en primera persona la diferència de rendiment entre diverses arquitectures de programari. Mentre que un servidor web estàndard obliga la CPU a estar constantment bloquejada comprovant l'arribada de nous clients (*polling*), l'ús de llibreries asíncrones ha demostrat com gestionar peticions web en segon pla, alliberant el 100% del temps del bucle principal per a altres tasques crítiques.
- **La memòria Flash com a sistema de fitxers (SPIFFS):** S'ha après a donar un ús avançat a la memòria interna del xip, anant més enllà de simplement guardar-hi el codi del programa. Formatejar-la com a sistema SPIFFS ha permès crear un gestor d'arxius autònom capaç de rebre, llistar i lliurar documents a través de la xarxa.
- **La importància crítica del codi no bloquejant (OTA):** L'exercici d'actualització *Over-The-Air* ha consolidat definitivament el concepte del temps real en programació. S'ha evidenciat que instruccions com `delay()` són fatals per a dispositius connectats, ja que bloquegen el processador i provoquen la pèrdua d'escolta dels serveis de xarxa. L'ús de `millis()` s'ha revelat com la tècnica estàndard per mantenir viu l'enllaç de comunicacions alhora que s'executen rutines periòdiques.

En definitiva, l'assoliment d'aquestes competències dota el programador de les eines professionals necessàries per dissenyar sistemes encastats moderns. Dominar l'OTA, els servidors asíncrons i la gestió de memòria SPIFFS és el que separa un simple projecte d'electrònica domèstica d'un autèntic producte IoT robust, autònom i actualitzable de forma remota.